



UNIVERSITÉ DE LA NOUVELLE CALÉDONIE
DÉPARTEMENT DES SCIENCES ET TECHNIQUES

Attaques par canaux auxiliaires avec ChipWhisperer

Auteurs :

GALLOUZE Yanis
PORCU Matteo

Tuteur :

RICHMOND Tania

Formation : Licence Informatique TREC7

Année universitaire : 2025

21 septembre 2025

Table des matières

1	Introduction	3
2	Installation et mise en place du matériel	4
2.1	Présentation de la carte ChipWhisperer Nano	4
2.2	Installation des dépendances logicielles	5
2.3	Connexion et configuration initiale	6
3	Premières expérimentations : SPA (Simple Power Analysis)	7
3.1	Principe d'une SPA	7
3.2	Test avec le mot de passe trouvé (analyse visuelle des traces)	7
3.3	Interprétation des résultats	8
3.4	Attaque sur le mot de passe	9
3.4.1	Explication de l'algorithme	10
4	Rappels théoriques : AES	12
4.1	Présentation générale du chiffrement symétrique	12
4.2	Structure de l'algorithme AES	12
4.3	Différences entre TinyAES et mbedTLS pour l'analyse	13
4.3.1	Comparaison	14
5	Attaque CPA (Correlation Power Analysis)	15
5.1	Principe de l'attaque par CPA	15

5.2	Mise en œuvre avec ChipWhisperer	15
5.2.1	Configuration initiale	16
5.2.2	Initialisation et connexion	16
5.2.3	Compilation et programmation du firmware	16
5.2.4	Capture des traces	16
5.3	Extraction de la clé AES	17
6	Difficultés rencontrées et solutions	21
6.1	Problèmes techniques (installation, compatibilité, matériel)	21
6.2	Interprétation des résultats et temps de calcul	23
7	Conclusion	24

1 Introduction

Nous avons choisi ce projet sur les attaques par canaux auxiliaires pour notre projet tutoré avec la plateforme *ChipWhisperer* parce que nous sommes passionnés par la cybersécurité et souhaitons développer nos compétences dans ce domaine. Bien que notre objectif ne soit pas de devenir des experts en cryptographie, nous voulions nous familiariser avec ce type d'attaques pour mieux comprendre comment les systèmes sécurisés peuvent être vulnérables.

Au fil de ce rapport, nous allons présenter la mise en place du matériel et des logiciels nécessaires, expliquer les concepts de base des attaques par consommation électrique et montrer comment on peut analyser les traces pour déduire des informations sensibles. L'objectif est avant tout pédagogique : comprendre le fonctionnement d'un microcontrôleur, apprendre à capturer et interpréter des données issues de son exécution, et se familiariser avec les notions fondamentales des attaques par canaux auxiliaires.

2 Installation et mise en place du matériel

2.1 Présentation de la carte ChipWhisperer Nano

La carte *ChipWhisperer Nano* constitue une plateforme matérielle spécialement conçue pour l'étude des attaques par canaux auxiliaires. Compacte et abordable, elle met à disposition des étudiants et chercheurs un environnement d'expérimentation permettant de capturer et d'analyser les traces de consommation électrique ou électromagnétique d'un microcontrôleur lors de l'exécution d'algorithmes cryptographiques. Elle intègre à la fois une cible programmable et un module de capture synchronisé, facilitant ainsi la mise en œuvre de techniques telles que la *Simple Power Analysis* (SPA) ou la *Correlation Power Analysis* (CPA). Grâce à son interface logicielle compatible avec Python et Jupyter, la carte offre une flexibilité d'utilisation et s'impose comme un outil pédagogique de référence pour l'apprentissage des concepts liés à la sécurité matérielle et aux attaques par canaux auxiliaires.

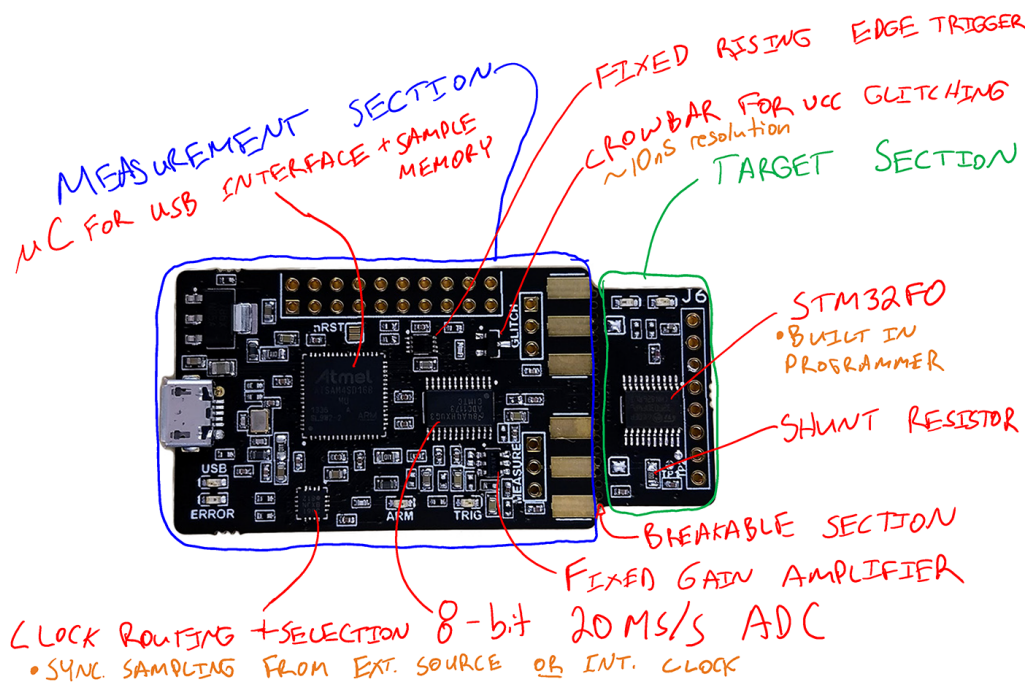


FIGURE 2.1 – Schéma annoté de la carte ChipWhisperer Nano

La figure 2.1 illustre la structure interne de la carte *ChipWhisperer Nano*, mettant en évidence ses principales sections fonctionnelles. On distingue tout d’abord la partie **Measurement Section**, qui comprend le microcontrôleur chargé de l’interface USB et de la mémoire d’échantillonnage, ainsi que l’ADC 8 bits capable d’atteindre une fréquence de 20 MS/s. Cette section est essentielle pour la capture et l’analyse des traces de consommation électrique de la cible. La **Target Section**, basée sur un microcontrôleur STM32F0, intègre un programmeur embarqué et un shunt resistor permettant de mesurer précisément le courant consommé lors de l’exécution d’algorithmes cryptographiques. Enfin, d’autres composants comme le système de *clock routing*, le gain fixe de l’amplificateur, ainsi que le module de glitching d’alimentation (*crowbar*) offrent la possibilité de réaliser des attaques avancées en synchronisant ou en perturbant le fonctionnement de la cible. Grâce à cette architecture, la carte combine à la fois un générateur de traces et une cible embarquée, ce qui en fait un outil compact et complet pour l’apprentissage et la recherche en sécurité matérielle.

2.2 Installation des dépendances logicielles

L’environnement de travail a été configuré à l’aide du sous-système Windows pour Linux (*Windows Subsystem for Linux* – WSL), afin de bénéficier d’un environnement compatible avec les outils nécessaires au fonctionnement de la carte *ChipWhisperer Nano*. Toutefois, avant de pouvoir exploiter WSL, il a été nécessaire d’effectuer une redirection de port via *PowerShell*, permettant à l’interface USB de la carte d’être correctement reliée au sous-système Linux. Cette opération s’est réalisée à l’aide des commandes suivantes :

```
usbipd wsl list
usbipd wsl attach --busid <ID>
usbipd bind --busid <ID>
```

Après avoir redirigé le périphérique USB vers WSL grâce à `usbipd`, l’ensemble des bibliothèques et dépendances a été installé à l’aide des commandes suivantes :

```
sudo apt update && sudo apt upgrade

sudo apt install make git avr-libc gcc-avr \
    gcc-arm-none-eabi libusb-1.0-0-dev usbutils python3 python3-venv python3-dev

cd ~/
git clone https://github.com/newaetech/chipwhisperer
cd chipwhisperer

python3 -m venv ~/.cwvenv
```

```
source ~/.cwvenv/bin/activate

sudo cp 50-newae.rules /etc/udev/rules.d/50-newae.rules
sudo udevadm control --reload-rules
sudo groupadd -f chipwhisperer
sudo usermod -aG chipwhisperer $USER
sudo usermod -aG plugdev $USER
git submodule update --init jupyter

python -m pip install -e .
python -m pip install -r jupyter/requirements.txt
```

Ces commandes permettent d'installer les compilateurs nécessaires (AVR et ARM), les dépendances USB, ainsi que l'environnement virtuel Python contenant l'API *ChipWhisperer* et les modules Jupyter. L'ajout de règles `udev` et la création d'un groupe spécifique assurent par ailleurs une gestion correcte des droits d'accès au périphérique USB.

Une fois cette configuration effectuée, l'environnement de travail peut être lancé à l'aide des commandes suivantes :

```
source ~/.cwvenv/bin/activate
cd ~/chipwhisperer
jupyter notebook
```

Cela permet d'ouvrir un serveur Jupyter directement connecté à la librairie *ChipWhisperer*, offrant ainsi un espace interactif pour programmer la carte.

2.3 Connexion et configuration initiale

Après l'installation des dépendances, la configuration de la carte *ChipWhisperer Nano* a été effectuée via *Jupyter Notebook* dans le dossier `jupyter`. Deux fichiers ont été lancés : *Introduction to Jupyter Notebooks*, utilisé pour se familiariser avec les commandes Python de base (`scope`, `target`, `plot`), puis *Connecting to Hardware*, permettant de vérifier la détection et le bon fonctionnement du matériel. Ces tests ont confirmé que la communication USB et l'initialisation de la carte étaient opérationnelles, rendant l'environnement prêt pour les expérimentations.

Ainsi, à l'issue de cette étape, la carte était prête à être utilisée pour les expérimentations liées aux attaques par canaux auxiliaires.

3 Premières expérimentations : SPA (Simple Power Analysis)

3.1 Principe d'une SPA

La *Simple Power Analysis* (SPA) est une méthode d'attaque par canaux auxiliaires qui consiste à observer directement la consommation électrique d'un dispositif lors de l'exécution d'un algorithme cryptographique. L'idée est que certaines opérations, comme les accès mémoire ou les calculs liés aux clés, produisent des variations caractéristiques dans la courbe de consommation. En analysant visuellement ces traces, il devient possible de distinguer les étapes de l'algorithme et parfois de déduire des informations sensibles, par exemple la longueur d'un mot de passe ou certaines parties d'une clé secrète. Contrairement à la CPA, la SPA repose surtout sur l'interprétation manuelle de signaux plutôt que sur des calculs statistiques complexes.

3.2 Test avec le mot de passe trouvé (analyse visuelle des traces)

Lors de l'expérimentation de la *Simple Power Analysis*, la carte *ChipWhisperer Nano* a été utilisée pour exécuter un programme de vérification de mot de passe. À chaque tentative, la consommation électrique du microcontrôleur a été enregistrée et représentée sous forme de courbe.

L'observation de ces traces a permis d'identifier des différences visibles selon que le mot de passe entré était correct ou erroné. En particulier, certaines variations dans la forme du signal apparaissaient uniquement lorsque le mot de passe correspondait à celui attendu par la cible.

Cette analyse visuelle a confirmé la possibilité de distinguer le comportement du système sans accéder directement au code source, démontrant ainsi l'efficacité d'une attaque SPA même sur un exemple simple.

Algorithm 1: Pseudocode de la fonction de capture après reboot

Input: Caractère testé `guessed_char`, scope, target
Output: Trace capturée
`capture_trace_after_password(guessed_char)` Redémarrer la cible ;
 Armer le scope pour la capture ;
 Envoyer le caractère testé à la cible ;
 Lancer la capture ;
 Récupérer la dernière trace capturée ;
 Retourner ;

3.3 Interprétation des résultats

La figure 3.1 montre la superposition des trois courbes obtenues :

- en bleu, la trace correspondant à la lettre correcte (**h**) ;
- en rouge et en jaune, les traces correspondant aux hypothèses erronées (**a** et **b**).

On observe que la courbe bleue présente des motifs réguliers, visibles à plusieurs instants précis de la capture. Ces variations sont caractéristiques des opérations de chiffrement déclenchées par l'entrée du bon premier caractère. En comparaison, les courbes rouge et jaune ne présentent pas ces structures : elles apparaissent plus aléatoires et identiques à elles-mêmes, ce qui indique qu'aucun traitement significatif lié au mot de passe n'a eu lieu.

Ainsi, l'interprétation visuelle du graphe confirme que la lettre **h** est bien la bonne hypothèse, tandis que **a** et **b** sont rejetées.

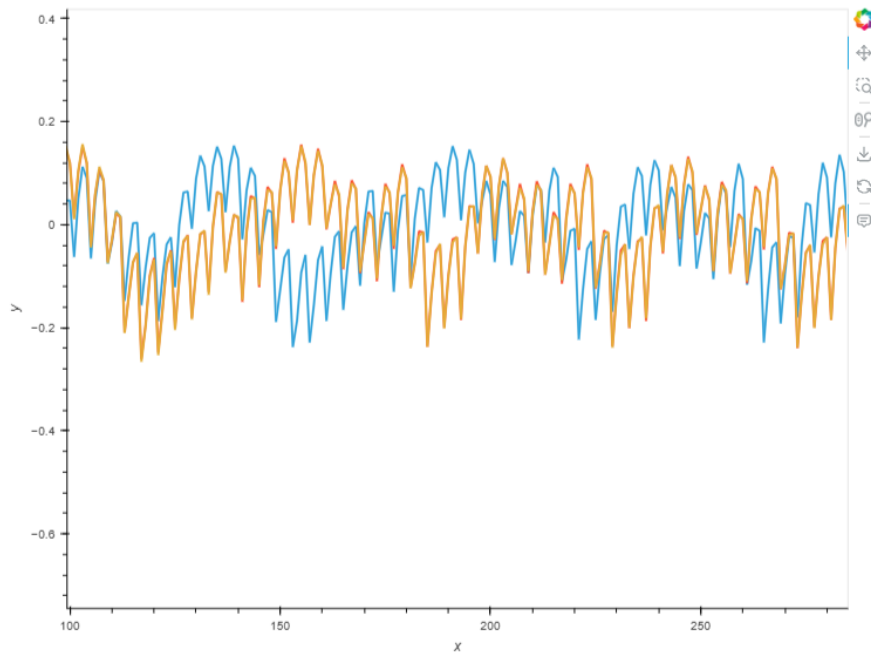


FIGURE 3.1 – Comparaison des traces pour trois hypothèses de lettres (h, a, b).

3.4 Attaque sur le mot de passe

Algorithm 2: Partie 1 : initialisation et calcul des différences

Input: Charset de caractères, fonction
 capture_trace_after_password(chaine), seuil_diff

Output: Mot de passe deviné (progressif) guessed_pw

Initialiser guessed_pw = "";

Initialiser charset = "abcdefghijklmnopqrstuvwxyz0123456789";

while True **do**

 Obtenir la trace de référence après envoi de guessed_pw + "\x00" :
 ref_trace;

 Initialiser max_diff $\leftarrow -\infty$ et best_char $\leftarrow$ None;

foreach caractère c dans charset **do**

 Capturer la trace pour guessed_pw + c : trace;

 Calculer la différence : diff $\leftarrow \sum |trace - ref_trace|$;

 Afficher la paire testée et la différence (guessed_pw + c $\rightarrow$ diff);

end

end

Algorithm 3: Partie 2 : sélection du meilleur caractère et continuation

```

while True do
  if diff > max_diff then
    | Mettre à jour max_diff  $\leftarrow$  diff et best_char  $\leftarrow$  c;
  end
  if max_diff < seuil_diff then
    | Afficher [*] Fin détectée : aucun caractère ne provoque de
      | saut de consommation.;
    | stop;
  end
  Concaténer le meilleur caractère choisi :
  | guessed_pw  $\leftarrow$  guessed_pw + best_char;
  Afficher [+] Mot de passe partiel : guessed_pw;
end

```

3.4.1 Explication de l'algorithme

Idée générale L'algorithme essaye de trouver le mot de passe caractère par caractère :

1. On envoie le mot de passe actuel + un caractère de test.
2. On mesure la consommation électrique de la cible.
3. On compare cette trace avec une trace de référence.
4. Plus la différence est grande, plus le caractère testé a de chances d'être le bon.
5. On garde le meilleur caractère et on recommence pour la position suivante.
6. Si aucune différence n'est assez grande, on arrête : le mot de passe est trouvé.

Exemple

- Première boucle : seul le caractère **h** donne une différence très forte $\rightarrow$ on choisit **h**.
- Deuxième boucle : avec **h** comme début, c'est **0** qui donne le plus grand pic $\rightarrow$ mot de passe **h0**.
- Ensuite on trouve **p**, puis **x**, puis **3**.
- On obtient donc progressivement : **h** $\rightarrow$ **h0** $\rightarrow$ **h0p** $\rightarrow$ **h0px** $\rightarrow$ **h0px3**.
- Après **h0px3**, une dernière forte différence indique encore un caractère correct, ici **b**.

À retenir

- Une grande différence dans les traces signifie que le caractère est correct.
- Le seuil sert à ignorer les petites variations dues au bruit.

- Si on connaît déjà la longueur du mot de passe, on peut simplement répéter la boucle autant de fois que nécessaire (ce qu'on a fait plus tard car on a triché en regardant le code où le mot de passe était :-)).

4 Rappels théoriques : AES

4.1 Présentation générale du chiffrement symétrique

Le chiffrement symétrique est une méthode de cryptographie reposant sur l'utilisation d'une même clé pour le chiffrement et le déchiffrement des données.

L'expéditeur transforme un message en texte chiffré à l'aide d'un algorithme et d'une clé secrète partagée, et le destinataire applique le même algorithme avec cette clé pour retrouver le message original. Ce modèle est largement utilisé en raison de sa rapidité et de son efficacité, notamment pour le traitement de grands volumes de données. Cependant, sa principale faiblesse réside dans la gestion et l'échange sécurisé de la clé, qui doit rester confidentielle entre les deux parties. Parmi les algorithmes les plus connus de chiffrement symétrique, on retrouve l'AES et le DES (ancêtre de l'AES, son emploi n'est plus recommandé aujourd'hui, du fait de sa lenteur à l'exécution et de son espace de clés trop petit de 56 bits permettant une attaque en un temps raisonnable).

4.2 Structure de l'algorithme AES

L'algorithme AES (*Advanced Encryption Standard*) est un chiffrement symétrique par blocs qui opère sur des blocs de 128 bits, avec des clés de 128, 192 ou 256 bits. Son fonctionnement repose sur une série de transformations répétées, appelées *rounds*, dont le nombre dépend de la taille de la clé (10, 12 ou 14 tours). Chaque round applique successivement plusieurs opérations :

- **SubBytes** : une substitution non linéaire est appliquée à chaque octet à l'aide de la S-Box, renforçant la confusion du système.
- **ShiftRows** : une transposition qui décale les lignes de l'état afin de disperser les dépendances entre octets.
- **MixColumns** : une transformation linéaire opérant sur les colonnes pour diffuser l'information entre les octets.
- **AddRoundKey** : chaque octet de l'état est combiné par un XOR avec une sous-clé dérivée de la clé principale.

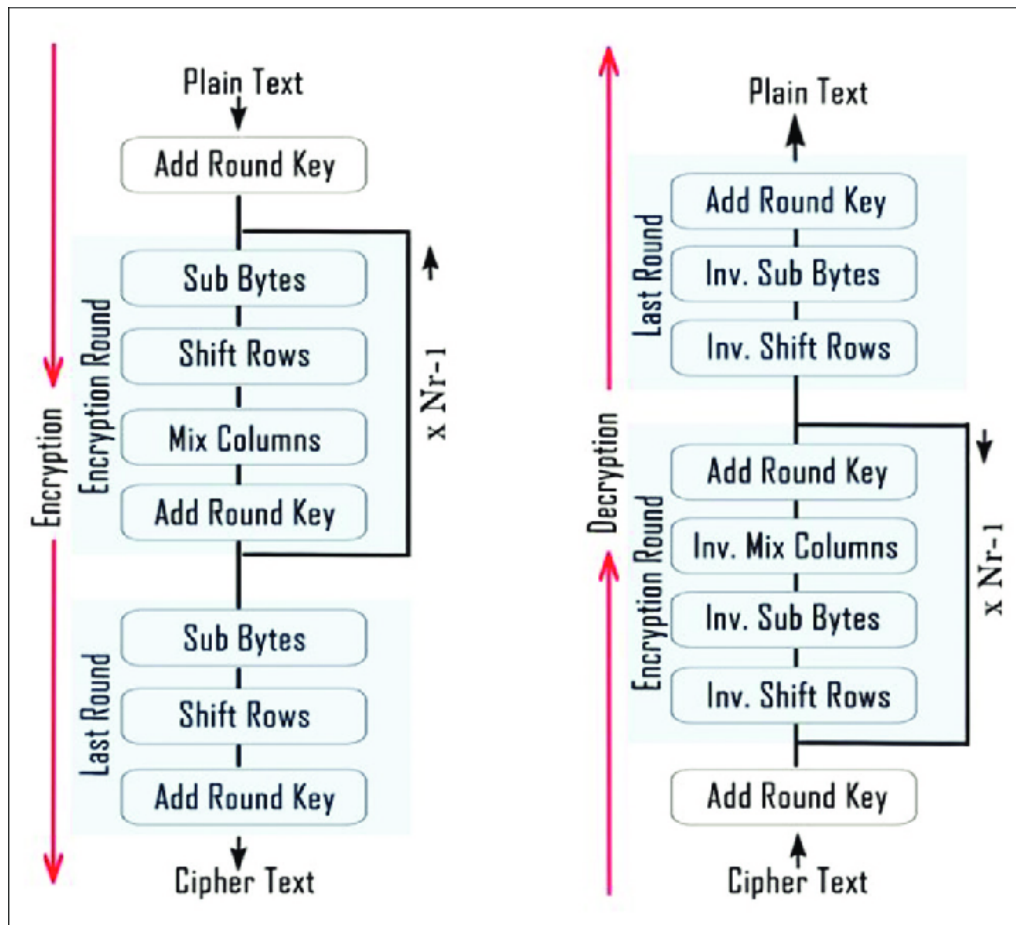


FIGURE 4.1 – Schéma d'un tour d'AES

Ces étapes, appliquées de manière répétée, assurent à la fois confusion et diffusion, garantissant la robustesse d'AES face aux attaques classiques.

4.3 Différences entre TinyAES et mbedTLS pour l'analyse

TinyAES est une implémentation légère de l'algorithme AES, conçue pour être simple, lisible et facilement intégrable dans des systèmes embarqués. Elle met en avant la clarté du code plutôt que l'optimisation, ce qui permet une meilleure visibilité des opérations internes lors de l'exécution.

Tandis que **mbedTLS** est une bibliothèque de cryptographie plus complète et optimisée, développée pour fournir des performances élevées et une large gamme de protocoles de sécurité. Son implémentation d'AES utilise des optimisations telles que les tables de substitution (T-tables) ou l'usage d'instructions matérielles lorsque disponibles, afin de maximiser la rapidité.

Nous avons utilisé ces deux implémentations d'AES car elles sont les plus recommandées et les plus légères.

4.3.1 Comparaison

- **Complexité du code** : TinyAES est minimaliste et expose clairement chaque tour de l’AES, tandis que mbedTLS masque ces détails dans une structure plus optimisée et modulaire.
- **Rapidité d’exécution** : mbedTLS est nettement plus rapide grâce à ses optimisations (T-tables, instructions spécifiques), alors que TinyAES est volontairement plus lent.
- **Analyse par canaux auxiliaires** : avec TinyAES, les tours de chiffrement sont visibles dans les traces, ce qui facilite les attaques par corrélation. Avec mbedTLS, l’optimisation rend les motifs moins nets, ce qui complique l’identification et demande plus de traces.

5 Attaque CPA (Correlation Power Analysis)

5.1 Principe de l'attaque par CPA

L'attaque par *Correlation Power Analysis* (CPA) cherche à retrouver la clé en comparant des hypothèses de consommation à la consommation réelle mesurée. Avec la *ChipWhisperer Nano*, le procédé se déroule généralement ainsi : on capture un grand nombre de traces de consommation pendant l'exécution du chiffrement (en configurant le **scope** : gain, fréquence d'échantillonnage, trigger), puis on choisit pour chaque octet de clé candidat k une valeur intermédiaire calculée à partir du plaintext et de k (par exemple la sortie de la S-Box). On applique un modèle de fuite pour transformer cette valeur intermédiaire en une hypothèse de consommation. Pour chaque échantillon temporel, on calcule la corrélation entre le vecteur des consommations mesurées (pour toutes les traces) et le vecteur des consommations prédites pour la clé candidate. La clé la plus probable est celle dont la candidate atteint la corrélation la plus élevée en valeur absolue sur au moins un échantillon. Le succès dépend du nombre de traces, de la qualité de l'acquisition et des réglages matériels sur la Nano.

5.2 Mise en œuvre avec ChipWhisperer

Le script Python utilisé pour ce projet permet de configurer et piloter le scope *ChipWhisperer* afin de capturer des traces de consommation électrique lors d'opérations de chiffrement AES. Il peut fonctionner en deux modes : **HARDWARE**, où la cible est programmée et les traces sont collectées directement depuis la carte, et **SIMULATED**, où un projet existant de traces est ouvert pour analyse. L'objectif principal est de préparer des données exploitables pour des attaques par canaux auxiliaires, notamment les attaques par corrélation (CPA).

5.2.1 Configuration initiale

Les paramètres principaux du script incluent :

- `SCOPE_TYPE`, `PLATFORM`, `VERSION` : définissent le type de scope, la carte cible et le mode de fonctionnement.
- `SS_VER` : version du protocole SimpleSerial.
- `CRYPTO_TARGET` : implémentation AES utilisée (TinyAES ou mbedTLS). TinyAES expose clairement les tours de chiffrement, facilitant l'analyse, tandis que mbedTLS est optimisé et plus rapide mais rend l'identification des tours plus complexe.

5.2.2 Initialisation et connexion

En mode `HARDWARE`, le script :

- Importe le module `chipwhisperer` et crée ou connecte le scope.
- Sélectionne le type de cible (`SimpleSerial2` ou fallback vers `SimpleSerial`) selon la version du protocole.
- Connecte la cible avec gestion des erreurs USB et reconnexion si nécessaire.
- Détecte le programmeur adapté à la carte (STM32, XMEGA, SAM4S, etc.) et effectue un reset matériel approprié.

5.2.3 Compilation et programmation du firmware

Le firmware AES (`simpleserial-aes`) est compilé avec `make` pour la plateforme choisie et programmé sur la cible via `cw.program_target`. Les erreurs de compilation sont capturées dans un fichier temporaire pour diagnostic.

5.2.4 Capture des traces

Un générateur de paires clé/texte (`ktp = cw.ktp.Basic()`) produit les données d'entrée. Pour chaque paire :

- Une trace de consommation est capturée avec `cw.capture_trace(scope, target, text, key)`.
- Si valide, la trace est ajoutée au projet `.cwp`.

Le nombre total de traces capturées est défini par l'utilisateur (`N`). Après capture, le projet est sauvegardé et le scope ainsi que la cible sont déconnectés proprement.

Algorithm 4: Pseudocode pour la capture des traces AES avec ChipWhisperer

Input: Nombre de traces N , scope et cible ChipWhisperer

Output: Projet contenant les N traces capturées

Initialiser le générateur de paires clé/texte ;

Créer un projet pour stocker les traces;

for $i = 1$ à N **do**

 Générer une paire clé/texte ;

 Capturer la trace ;

if *trace valide* **then**

 Ajouter la trace au projet ;

end Optionnel

 Afficher le compteur de déclenchement de l'ADC;

end

Sauvegarder le projet;

Déconnecter le scope et la cible proprement ;

5.3 Extraction de la clé AES

Une fois les traces capturées, il est possible d'analyser la consommation pour tenter de récupérer la clé AES. Le script permet de visualiser la consommation d'énergie pendant le chiffrement grâce à la commande :

```
cw.plot(project.waves[0])
```

Avec **TinyAES**, chaque tour de chiffrement (SubBytes, ShiftRows, MixColumns, AddRoundKey) est exécuté séparément, ce qui crée des motifs réguliers visibles dans la consommation et facilite la détection des tours. En revanche, avec **mbedTLS**, l'implémentation est optimisée : les opérations sont fusionnées et alignées en mémoire, rendant les tours moins lisibles sur le graphe.

Algorithm 5: Pseudocode pour l'attaque par corrélation (CPA)

Input: Projet contenant les traces AES, modèle de fuite (ex : S-box ou T-tables)

Output: Clé AES estimée

Sélectionner le modèle de fuite adapté;

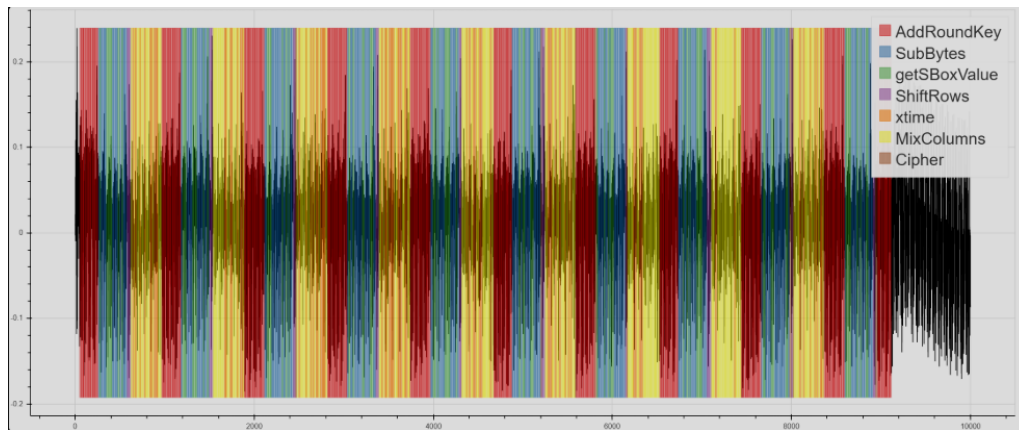
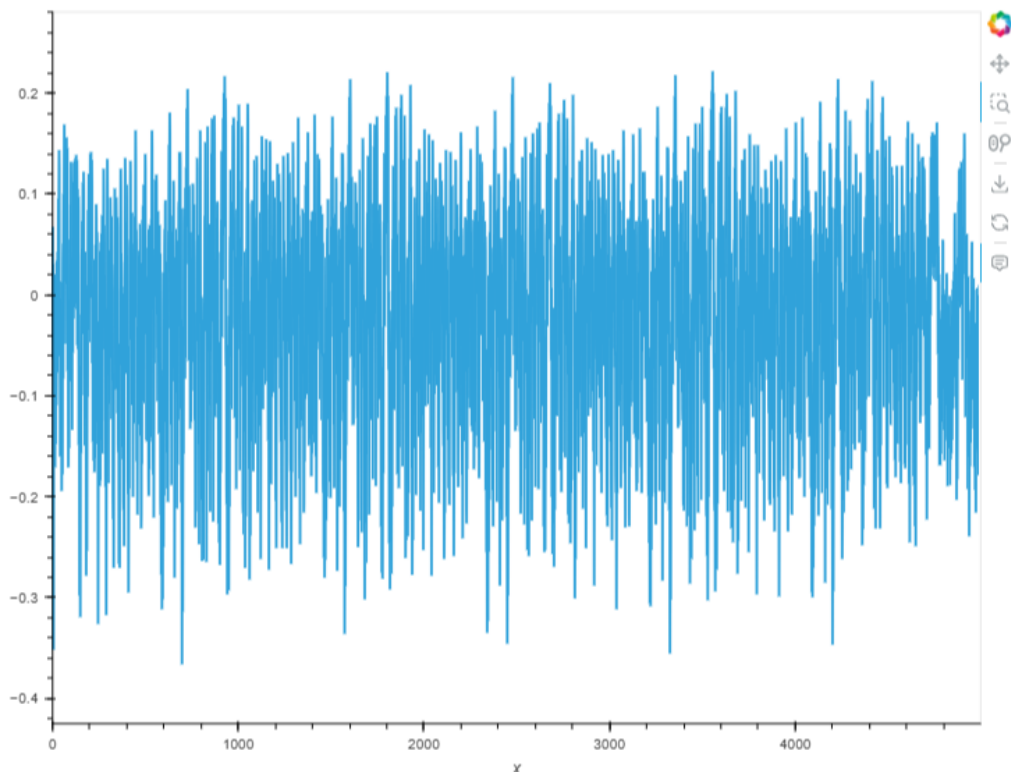
Créer une attaque CPA sur le projet avec ce modèle;

Exécuter l'attaque et récupérer les résultats;

Analyser les résultats pour estimer la clé AES;

Si nécessaire, on peut remplacer le modèle S-box par un modèle basé sur les T-tables.

On a exécuté cette attaque de démonstration avec mbedTLS car il est plus rapide

FIGURE 5.1 – Exemple de trace obtenue avec **TinyAES**.FIGURE 5.2 – Exemple de trace obtenue avec **mbedTLS** montrant l'illisibilité des traces.

pour gagner du temps.

Comment le tableau permet de retrouver la clé ?

- Pour chaque valeur possible de l'octet de clé (0 à 255), on calcule la sortie de la S-box en fonction du texte clair connu.
- On compare ces valeurs avec les traces de consommation mesurées.
- Pour chaque hypothèse, un score est calculé (ici affiché sous la valeur hexa-

décimale).

— L’hypothèse qui donne le score le plus élevé (≤ 1) est celle qui correspond à la vraie clé.

Même si mbedTLS va plus vite que TinyAES, le principe reste le même : on teste toutes les clés possibles, on regarde celle qui « colle » le mieux aux traces, et le tableau montre clairement quel octet de clé est correct.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PGE=	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	2B 0.834	7E 0.473	15 0.705	16 0.846	28 0.885	AE 0.877	D2 0.743	A6 0.735	AB 0.816	F7 0.817	15 0.785	88 0.722	09 0.782	CF 0.893	4F 0.814	3C 0.856
1	84 0.468	BF 0.456	A4 0.496	50 0.491	51 0.459	31 0.473	6E 0.482	D1 0.489	B3 0.473	C1 0.467	12 0.467	E7 0.467	D2 0.489	29 0.461	7B 0.462	EF 0.538
2	B1 0.447	5F 0.455	C9 0.488	07 0.455	86 0.457	CD 0.467	D0 0.459	4A 0.453	CB 0.466	19 0.458	D4 0.466	C8 0.456	71 0.469	E1 0.459	CB 0.461	05 0.453
3	E9 0.447	02 0.450	CD 0.478	8E 0.453	E4 0.454	64 0.465	56 0.458	FC 0.441	5B 0.452	08 0.458	D5 0.460	77 0.454	59 0.464	98 0.450	09 0.458	56 0.450
4	71 0.445	A8 0.444	58 0.476	49 0.450	2F 0.449	E7 0.448	5C 0.446	58 0.440	1B 0.445	C7 0.441	30 0.451	23 0.444	7D 0.462	EC 0.448	E0 0.446	AA 0.450

FIGURE 5.3 – Tableau résultant de l’attaque

Implémentation	Principe	Impact sur l'analyse
S-box	- Chaque octet est passé dans une table de substitution (256 valeurs). - Étapes séparées : Sub-Bytes puis MixColumns.	- Traces plus nettes et structurées. - Les pics de consommation correspondent aux opérations sur la S-box. → Plus "facile" à attaquer.
T-tables	- Utilisation de 4 tables pré-calculées (T0, T1, T2, T3). - Chaque entrée combine déjà S-box + MixColumns. - Réduction du nombre d'opérations.	- Traces plus « lissées », pics moins distincts. - Fuites plus difficiles à exploiter. → Plus rapide, mais demande plus de traces pour réussir une attaque CPA.

TABLE 5.1 – Comparaison entre implémentation AES avec S-box et T-tables

6 Difficultés rencontrées et solutions

6.1 Problèmes techniques (installation, compatibilité, matériel)

Au cours du projet, plusieurs difficultés techniques ont été rencontrées.

Dans un premier temps, l'installation des dépendances Python pour ChipWhisperer a posé problème : certaines erreurs apparaissaient lors de l'exécution du code, ce qui a conduit à une réinstallation complète de l'environnement sur WSL.

Un autre obstacle concernait la détection de la carte par WSL, qui ne reconnaissait pas correctement le périphérique USB. Ce problème a été résolu grâce à la mise en place d'un *bind* du port USB, permettant une communication correcte entre la carte et l'environnement virtuel.

Vers la fin du projet, des erreurs de type *data loss* sont également survenues sur la petite ChipWhisperer Nano. Ce problème a persisté lors de la rédaction du rapport : nous souhaitions prendre des captures d'écran des graphes et autres résultats, mais l'exécution du code restait compromise par ces pertes de données. Nous avons tenté de downgrader le firmware plusieurs fois, sans succès.

Nous avons donc décidé de prendre en main la ChipWhisperer Lite. Contrairement à la Nano, cette carte est une version dédiée à la capture, elle ne dispose pas de cible intégrée. Cela a nécessité de consulter beaucoup de documentation qui "n'existe plus" (qu'on a trouvée à l'aide d'Internet Archive) pour comprendre son fonctionnement. Finalement, nous avons réussi à la configurer avec une CW308 UFO et un shield STM32F3, ce qui nous a permis de poursuivre la rédaction du rapport et d'obtenir des captures fiables.

Même avec ce combo CW Lite + CW308 STM32F3, des warnings de *data loss* sont parfois apparus, mais ceux-ci ne sont que des avertissements : ils n'altèrent pas les résultats ni l'exécution du code, contrairement à la Nano où les data loss rendaient toute acquisition impossible.

Update : problème résolu ! Plus de data loss, il fallait juste mettre la bonne version du firmware avant de flasher la target de la carte : *SS VER 2 1*. Un simple petit

oubli nous a presque coûté cher !

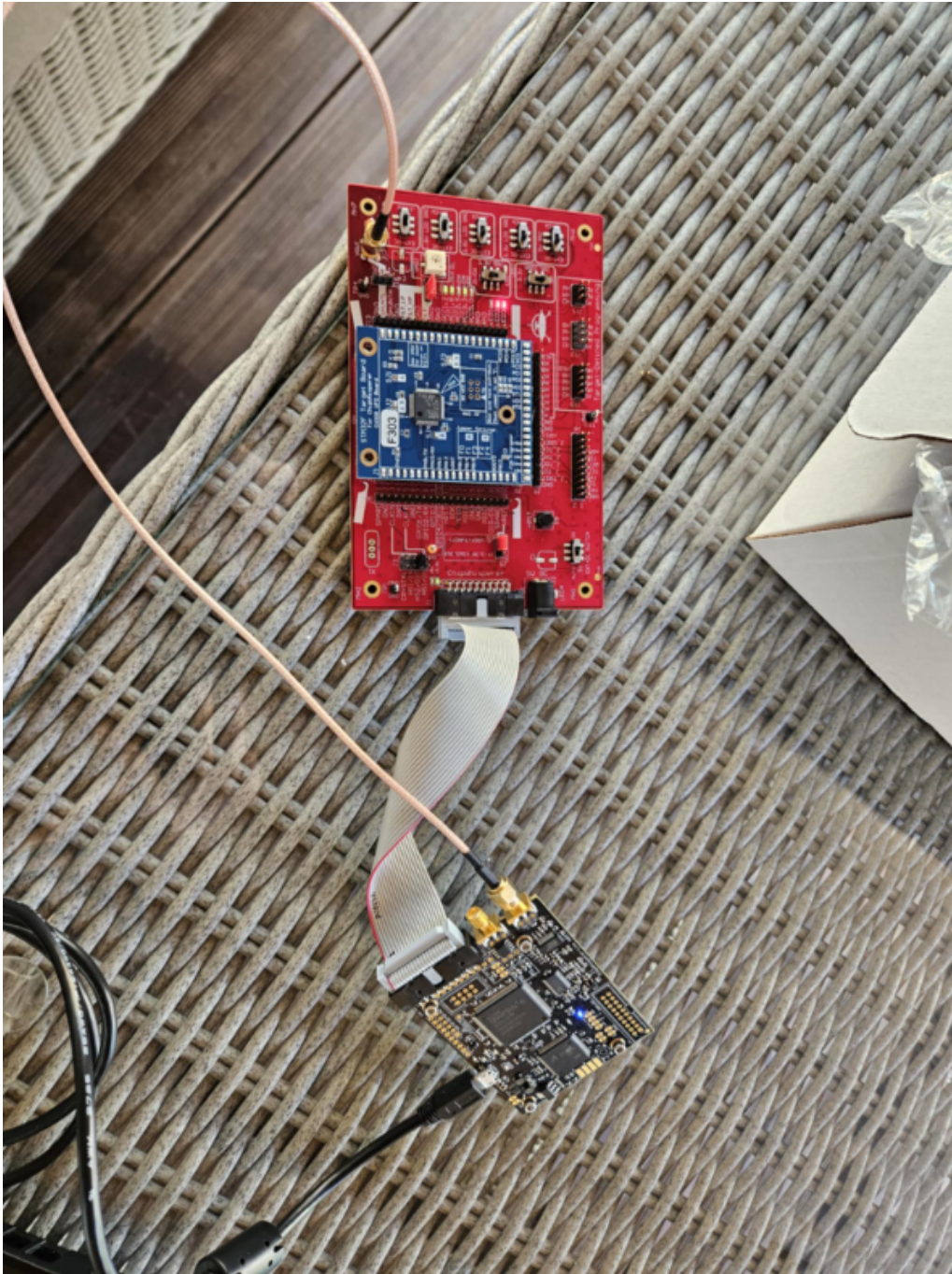


FIGURE 6.1 – Le setup CW308 UFO + CWLITE (Appelé souvent CW308-STM32F3)

6.2 Interprétation des résultats et temps de calcul

L'interprétation des traces issues des attaques a demandé une phase d'apprentissage prolongée : il a fallu plusieurs jours d'analyse et la consultation de tutoriels pour maîtriser la lecture des courbes générées (par exemple celles obtenues avec TinyAES, où les différents tours d'AES deviennent identifiables sur le signal). Nous avons progressivement appris à repérer les motifs liés aux opérations de l'AES.

7 Conclusion

Ce projet nous a permis de découvrir concrètement comment des attaques par canaux auxiliaires peuvent être réalisées et analysées. Nous avons appris à configurer le matériel, installer les logiciels et interpréter les traces électriques d'un microcontrôleur, tout en restant à un niveau général et pédagogique.

Même avec toutes les difficultés que nous avons eues durant ce projet, cette expérience nous a donné une meilleure compréhension de la sécurité matérielle et des vulnérabilités possibles. Elle constitue ainsi une étape importante dans notre apprentissage de la cybersécurité et nous motive à approfondir nos connaissances dans ce domaine.

Bibliographie

- [1] ChipWhisperer Documentation, *ChipWhisperer-Husky*, <https://chipwhisperer.readthedocs.io/en/latest/Capture/ChipWhisperer-Husky.html>.
- [2] NewAE Technology, *SCAPACK-L2 Datasheet*, https://web.archive.org/web/20211202072554/http://media.newae.com/datasheets/NAE-SCAPACK-L2_datasheet.pdf.
- [3] ChipWhisperer Documentation, *ChipWhisperer-Lite*, <https://rtfm.newae.com/Capture/ChipWhisperer-Lite/>.
- [4] NewAE Wiki, *Programming ARM*, https://wiki.newae.com/Programming_Arm.
- [5] NewAE Documentation, *CW308T-STM32F Targets*, <https://web.archive.org/web/20200925232713/https://rtfm.newae.com/Targets/UFO%20Targets/CW308T-STM32F/>.
- [6] GitHub, *CWNANO Power Analysis for Password Bypass*, https://github.com/newaetech/ChipWhisperer-Test-Results/blob/main/tutorials/CWNANO/SOLN_Lab%202_1B%20-%20Power%20Analysis%20for%20Password%20Bypass.ipynb.
- [7] NewAE Documentation, *CW308 UFO Targets*, <https://web.archive.org/web/20201031002530/https://rtfm.newae.com/Targets/CW308%20UFO/>.
- [8] Common errors, Chipwhisperer Documentation, *Data Loss Problem*, https://chipwhisperer.readthedocs.io/en/latest/cw_tips_tricks/common_errors/target_errors.html.